

Types for Programs and Proofs

Introduction

Thierry Coquand

What this course is about

Programs that cannot go wrong; proofs that a machine can check. One answer: *dependent type theory*

- a type can mention a value: $\text{Vec } A \ n$, vectors of length n
- a proposition is a type, a proof is a program of that type
- checking a proof = type-checking a program

The aim of this course: explain *how proof checking using dependent types works*, using *Agda*, a proof assistant that is *nothing more* than this type theory

Plan of this introduction

These 45 minutes (Thierry): the *applications*

- in mathematics: checking proofs, and what happens now that proofs are produced by machines
- in software: checking programs, from a compiler to what AI now makes possible

Next (Andreas): *applications in programming*, functional programming with dependent types

Then the course proper: how a proof checker based on dependent types works, and how to use one

Two dichotomies (the map for today)

Informal

a proof written for people
“it is clear that. . .”

Validation

a referee reads it and believes it
depends on who reads

Formal

a proof in a fixed language
every step is a rule

Verification

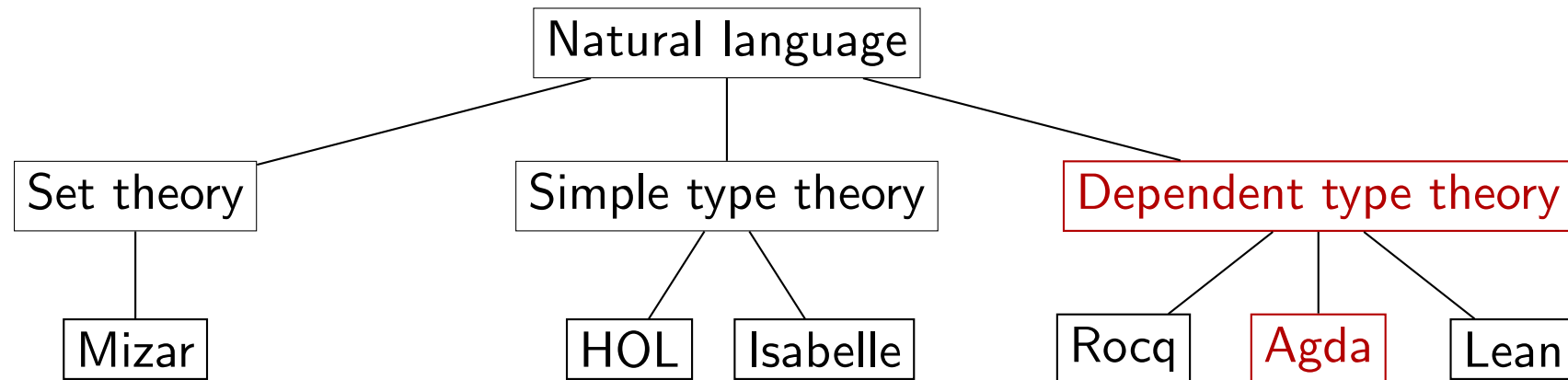
a small program checks every step
depends on what is checked

Mathematics: informal and validated. Programs: formal and unverified

Claim of this course: formal + verified is now practical, for both

The proof assistants

All of them formalise first-order logic; they differ on what comes next



First in the dependent branch: Automath (de Bruijn, 1968). This course: the right-hand branch, with Agda

The referee problem, 1879 – 2014

- Four colour: 1879 Kempe publishes a proof; 1890 Heawood finds the gap
- 1976 Appel–Haken: 1936 configurations checked by computer. No referee could check the whole
- 2005 Gonthier–Werner: the whole proof checked in Rocq
- Kepler: 1998 Hales: 300 pages + programs. Cohn: *“beyond the ability of the mathematics community to check thoroughly”*
- 2014 Flyspeck: formal proof completed (HOL Light, Isabelle)

Programs: the same problem, the same answer

CompCert (Leroy, Blazy, . . .): an optimising C compiler whose compilation passes come with proofs in Rocq. The main theorem:

```
transf_c_program p = OK tp  ->  backward_simulation (semantics p) (semantics tp)
```

Csmith (Yang, Chen, Eide, Regehr 2011), random testing, found hundreds of wrong-code bugs in gcc and clang. On CompCert:

the only compiler we have tested for which Csmith cannot find wrong-code errors . . . we have devoted about six CPU-years to the task

What exactly was proved?

Not “CompCert has no bugs”

A *precise statement* about the compiler is proved, and the proof is checked by a small *proof checker*: the kernel of Rocq

The statement means only what the formal semantics of C and of the processor mean. About 10% of the compiler (parsing, assembling, linking) is outside the proof

March 2026: CompCert qualified for the flight computer of the ATR 42/72 aircraft. The certification accepts the proof in place of some testing of the generated code

Proofs about computation: the Busy Beaver

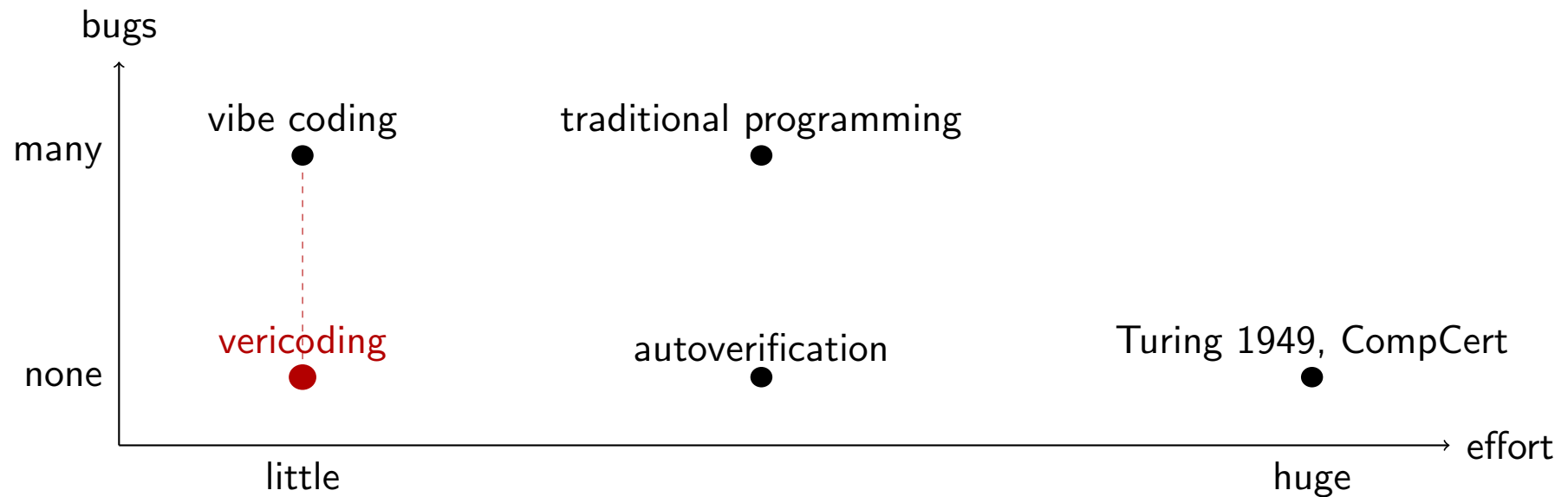
$BB(n)$: the longest run of a halting n -state Turing machine

bbchallenge (Stérin and many others): $BB(5) = 47\,176\,870$, proved in Rocq in 2024; $BB(6)$ is still open

- the proof is about programs and their runs: proofs must be able to *compute*
- much of the work was done by anonymous or student contributors
- their results were trusted because they were checked, not because of who they were

“Massively collaborative research projects have a bright future”

The history of coding (Tegmark, 2026)



Formal verification gave no bugs at a huge cost; AI removes the cost

From vibe coding to vericoding: who writes what?

	spec	code	proof	
traditional programming	–	human	–	bugs
vibe coding	–	AI	–	bugs
Turing 1949, CompCert	human	human	human	no bugs, huge effort
autoverification	human	human	AI	
<i>vericoding</i>	<i>human</i>	AI	AI	no bugs, little effort

2026: `gzip decompress ∘ compress = id` proved in Lean in a week (Morrison, with an AI); the Signal protocol being verified in Lean

Everything rests on the *spec*: that is what the human must write and read

2026: proofs faster than anyone wants to referee them

May 2026	Erdős unit-distance conjecture	false	Lean
Jul 2026	Grothendieck group-scheme conjecture	false	Lean, written in 4 hours
Jul 2026	Jacobian conjecture (1939)	false	computer algebra suffices
Aug 2026	Hopf: complex structure on S^6 (1947)	true	only Lean

In all four cases the mathematics was found with an AI

Reaction (K. Buzzard): no credit for an informal AI proof. The only question: *“is it formalised in Lean?”*

Prelude, Oct 2025: “GPT-5 solved 10 open Erdős problems” – it had found them in the literature. Everybody validated, nobody verified

S^6 , 23 – 27 August 2026: three files

Challenge.lean	1.6 KB	the <i>statement</i> , ending in sorry; written before any proof existed
Solution.lean	248 818 lines	the proof: 14 887 lemmas, mostly machine-written
config.json	248 B	the <i>auditor</i> : the theorem's name, the three axioms allowed

The informal paper (108 pages, Alpöge + Claude) is unrefereed

The formal proof (Alexeev + Codex) compiles

Belief comes from neither author: from a kernel of a few thousand lines,
trusting three axioms

The human's whole job: read this

```
import Mathlib
open scoped Manifold

abbrev unitSphere (n : Nat) :=
  Metric.sphere (0 : EuclideanSpace Real (Fin (n + 1))) 1

-- Does the 6-sphere admit a complex structure?
theorem mathoverflow_1973 :
  Exists atlas : ChartedSpace (EuclideanSpace Complex (Fin 3)) (unitSphere 6),
    letI := atlas
    IsManifold (modelWithCorners Complex (EuclideanSpace Complex (Fin 3))) 1
      (unitSphere 6) := by
sorry
```

Then check that the axiom list is the standard three, and build: about two minutes on a laptop

What is actually trusted

- the *statement*: does Challenge say what Hopf asked?
- the *axioms*: what was assumed without proof?
- the *kernel*: the type checker itself

All three are small enough for a human to read

The 248818 lines in between need not be read by anyone? One can argue that a good proof system should be able to *explain* the proof

In 1976 machine checking was a curiosity, in 2005 a reassurance; in 2026 it is what stands between “an AI said so” and “it is a theorem”

Autoformalisation

Autoformalisation: a machine translates informal mathematics (a paper, a textbook proof) into the language of a proof assistant

This is not specific to Lean. Two Agda examples from this year:

- *Church–Rosser for the λ -calculus* (Tait/Martin-Löf), following Barendregt’s thesis lemma by lemma: 6 files, 2 300 lines
- *Church/Guard’s 1963 proof of Gödel’s second incompleteness theorem*: 184 files, about 53 000 lines, no postulates

Human: the *statements* and conventions. Machine: the proofs. Agda: the check

Paper to Agda: the correspondence table

The λ -calculus is the core of every functional language; Church–Rosser says the order of evaluation does not matter. Both are taught in this course

Header of `Confluence.agda` (Barendregt, Appendix II, pp. 128–133)

paper	here
-----	-----
<code>>=1</code> (one step reduction)	<code>Par</code> (parallel reduction)
<code>>=</code> (reduction)	<code>Beta</code> (many step beta reduction)
<code>=</code> (convertibility)	<code>Conv</code>
Lemma 6 (inversion of <code>>=1</code>)	pattern matching (no lemma needed)
Lemma 7 (the diamond for <code>>=1</code>)	<code>Par-diamond</code>
Lemma 8	<code>Par-strip</code> , <code>ParStar-diamond</code> , <code>Beta-confluent</code>
Theorem 10	<code>church-rosser</code>

Curry–Howard: a proof is a functional program. This course explains how

Why Agda and not Lean? (1)

This course is about *how a proof assistant based on type theory works*

- Agda *is* the type theory: dependent functions, inductive families, universes, and almost nothing else in the way
- In Agda you write the *proof term* yourself, with holes, and watch the goal change
- In Lean a tactic script *generates* the term; you rarely see the object the kernel checks

Why Agda and not Lean? (2)

- Computation is visible: a proof about a program runs the program
- Syntax and semantics of programming languages, the second half of the course, is what Agda does best
- Agda is developed here in Göteborg, in a line of type theory implementations since the 1980s. Andreas is one of its main developers

What you learn transfers: Lean 4 has the same core

Whatever an AI emits still has to be a term of such a language, and somebody has to know what the checker is checking

The first published dependently typed proof

de Bruijn, *The mathematical language Automath*, 1969; *AUTOMATH, A Language for Mathematics*, 1973

Euclidean and reflexive relations are symmetric

euclidean: $R\ x\ y \rightarrow R\ x\ z \rightarrow R\ y\ z$ reflexive: $R\ x\ x$

The same discipline as S^6 , on five lines: a Challenge with a hole

```
{-# OPTIONS --safe #-}
```

```
euclidSym : (A : Set) (R : A -> A -> Set)
  -> ((x y z : A) -> R x y -> R x z -> R y z)    -- euclidean
  -> ((x : A) -> R x x)                          -- reflexive
  -> (x y : A) -> R x y -> R y x
euclidSym A R eucl refl x y p = ?
```

Filling the hole (live)

In the hole, Agda shows the goal $R\ y\ x$ and the hypotheses. We have `eucl`

$$x\ y\ x : R\ x\ y \rightarrow R\ x\ x \rightarrow R\ y\ x$$

```
euclidSym A R eucl refl x y p = eucl x y x p (refl x)
```

Now audit it, as for S^6 . A cheating solution:

```
postulate cheat : (A : Set) (R : A -> A -> Set) (x y : A) -> R x y
euclidSym A R eucl refl x y p = cheat A R y x
```

--safe rejects the postulate; an unfilled hole is rejected too

The proof is a program: with R instantiated, it *computes*

The course

- Lectures 1–3: Agda, simply typed programming, dependent types, proving: inductive predicates and natural deduction
- Lectures 4–11: operational semantics and type systems, bidirectional type checking, all formalised in Agda
- 5 homework assignments, student presentations, take-home exam
- Agda 2.8.0; *Dependent Types at Work*, *Verified Functional Programming in Agda*, Pierce, *Types and Programming Languages*

`teach-types.github.io/www`

Two paradigms (Tegmark)

	programs, proofs	neural networks
inference	fine	fine
learning	poor	<i>excellent</i> : programs form a continuous space
trust	<i>a proof checker</i>	inscrutable

Finding a proof is search; checking it is easy

So: let the neural network learn, then ask it for a program and a proof that it meets *your* spec, and check the proof

What you must understand: the *statement*, and the *checker*. Not the AI, not the proof

Why learn this now?

The examples today are the situation as it is in 2026: you should know about it. This course does *not* ask you to use AI

Proof assistants were interesting before AI and remain so: the four colour theorem, CompCert, *BB(5)* were done by people

Autoformalisation changes things quickly, and nobody knows how it will evolve. What does not change: a formal proof is a term of a type theory, and something has to check it. Learning its basics is

- interesting conceptually: logic, computation and programming as one subject
- likely to be useful, whatever the tools look like in five years

Three things to take away

- Whoever writes a proof, a person or a machine, what makes it a proof is a precise *statement* and a small program that checks
- In dependent type theory these are one activity: writing a type, and writing a program of that type
- In type theory, *types are also programs*: a statement can compute, and the checker computes while checking

Whatever proves a theorem for you, you still have to be able to *state* it, and to know what the checker is checking